

# 数理逻辑课程大作业实验报告

组长姓名      组员姓名 1      组员姓名 2      组员姓名 3      组员姓名 4

2026 年 7 月 6 日

## 摘要

本文报告围绕 NesyLink 数理逻辑课程项目展开，目标是在 Zelda-like 地牢环境中设计可运行的 Agent，并对环境或策略中的关键部分进行 Lean 形式化建模与证明。

本项目已对 `task_1` 至 `task_5` 全部完成 Lean 环境形式化与可完成性证明，对应文件位于 `student_agent/lean/` 目录下。五个形式化模型均通过 Lean LSP 诊断（零错误），并遵循统一证明骨架：局部前置/后置引理  $\rightarrow$  必要性引理  $\rightarrow$  `Reachable` 归纳不变式  $\rightarrow$  `completion_postcondition`  $\rightarrow$  witness 计划  $\rightarrow$  可完成性。其中 `task_4` 因涉及旋转桥、条件门、装备获取与事件驱动显现机制，作为重点展开对象。此外，本文对 `task_5` 进一步补充了高层符号策略形式化：定义 `PolicyStage`、`symbolicPolicy`、`ActionEnabled` 和策略执行轨迹，并证明该策略从初态出发的 10 步执行轨迹动作合法且到达全宝箱开启目标。

## 目录

|          |                  |          |
|----------|------------------|----------|
| <b>1</b> | <b>项目背景与任务要求</b> | <b>3</b> |
| 1.1      | 课程作业目标           | 3        |
| 1.2      | 环境限制与评测要求        | 3        |
| 1.3      | 本文结构             | 3        |
| <b>2</b> | <b>环境与任务分析</b>   | <b>3</b> |
| 2.1      | NesyLink 环境概述    | 3        |
| 2.2      | 任务拆解             | 4        |
| 2.3      | 主要难点             | 4        |
| <b>3</b> | <b>总体方法设计</b>    | <b>4</b> |
| 3.1      | 系统框架             | 4        |
| 3.2      | 为什么选择该路线         | 5        |

|                             |           |
|-----------------------------|-----------|
| 目录                          | 2         |
| <b>4 视觉与状态抽取</b>            | <b>5</b>  |
| 4.1 输入与输出设计                 | 5         |
| 4.2 实现方法                    | 6         |
| 4.3 误差与局限                   | 6         |
| <b>5 策略与规划</b>              | <b>6</b>  |
| 5.1 基线策略                    | 6         |
| 5.2 多任务扩展                   | 7         |
| 5.3 最终提交版本                  | 7         |
| <b>6 Lean 形式化与证明</b>        | <b>7</b>  |
| 6.1 形式化对象总览                 | 7         |
| 6.2 Task 1: 单房间钥匙门形式化       | 8         |
| 6.3 Task 2: 战斗与条件门形式化       | 9         |
| 6.4 Task 3: 多房间往返形式化        | 11        |
| 6.5 Task 4: 旋转桥与隐藏宝箱形式化     | 12        |
| 6.6 Task 5: 四房间按钮门与全宝箱收集形式化 | 14        |
| 6.6.1 环境建模                  | 14        |
| 6.6.2 状态空间与转移函数             | 14        |
| 6.6.3 核心定理                  | 15        |
| 6.6.4 归纳不变式                 | 15        |
| 6.6.5 Witness 计划            | 15        |
| 6.6.6 策略形式化与正确性证明           | 15        |
| 6.7 证明统一思路                  | 16        |
| 6.8 证明局限与边界                 | 17        |
| <b>7 实验设计与结果分析</b>          | <b>17</b> |
| 7.1 实验设置                    | 17        |
| 7.2 实验结果                    | 18        |
| 7.3 结果分析                    | 19        |
| <b>8 项目分工与推进过程</b>          | <b>19</b> |
| 8.1 成员分工                    | 19        |
| 8.2 时间安排                    | 20        |
| <b>9 总结与展望</b>              | <b>20</b> |

# 1 项目背景与任务要求

## 1.1 课程作业目标

本次课程作业要求小组围绕给定游戏环境完成三部分内容：

1. 设计并实现一个能够完成任务的 Agent；
2. 使用 Lean 对环境或策略中的关键部分进行形式化建模；
3. 证明若干性质，例如动作合法性、安全性、可达性或策略正确性。

## 1.2 环境限制与评测要求

- 训练和调试阶段可以使用像素、reward 和 info 等信息。
- 最终测试阶段，Agent 不能直接依赖环境内部 info。
- 最终测试阶段，Agent 只能根据像素观测、reward 和物品栏信息进行决策。
- 如果使用机器学习方法，需要说明训练方式、信息来源以及与最终评测接口的一致性。

## 1.3 本文结构

本文后续将依次介绍环境与任务分析、总体方法设计、视觉与状态抽取、策略与规划、Lean形式化与证明（覆盖 task\_1 至 task\_5）、实验结果以及项目分工。

# 2 环境与任务分析

## 2.1 NesyLink 环境概述

NesyLink 是一个 Zelda-like 的离散动作地牢环境。默认任务接口采用 Gym 风格：`obs, info = env.reset(seed)`，以及 `obs, reward, terminated, truncated, info = env.step(action)`。本次课程任务要求最终提交版策略在推理阶段只能使用像素观测、reward 和显式提供的物品栏信息，不能直接读取 info 中的隐藏状态。

动作空间为 7 个离散动作：WAIT、UP、DOWN、LEFT、RIGHT、BUTTON\_A、BUTTON\_B。默认像素控制下，房间地图区域大小为  $160 \times 128$  像素，对应  $10 \times 8$  个 tile，每个 tile 为  $16 \times 16$  像素。默认观测 obs 为 RGB 图像，shape 为 (128, 160, 3)。环境还提供结构化 info 作为调试与监督接口，其中包含房间编号、角色坐标、事件记录、动态对象状态和奖励信号等信息。

五个任务的差异主要体现在关键机制上：`task_1` 是静态单房间取钥匙开门；`task_2` 引入怪物、战斗和条件门；`task_3` 要求跨房间记忆；`task_4` 引入旋转桥、条件门、装备获取和隐藏宝箱，是最适合做显式环境建模的一关；`task_5` 则是四房间按钮门与全宝箱收集的综合任务。基于课程要求与当前项目进度，本文对 `task_1` 至 `task_5` 全部进行了 Lean 环境形式化与可完成性证明，其中 `task_4` 因机制最复杂作为重点展开对象。

## 2.2 任务拆解

五个任务可以按关键机制逐步拆解：

- Task 1: 单房间取钥匙开门。
- Task 2: 怪物处理与取钥匙。
- Task 3: 多房间状态记忆与往返。
- Task 4: 桥与按钮机制、复杂子任务链。
- Task 5: 综合关卡与泛化挑战。

## 2.3 主要难点

1. 最终测试不能直接读取 `info`。
2. 需要从像素中抽取对策略有用的状态表示。
3. 需要在短时间内兼顾实现、证明和报告。

# 3 总体方法设计

## 3.1 系统框架

本项目采用“像素感知 + 规则状态机 + Lean 符号层验证”的整体框架。运行时，`student_agent/baseline_policy.py` 提供统一的 Policy 入口，评测脚本仍按 `act(obs, info)` 调用，但策略主体尽量只使用像素观测、reward 信号、物品栏和内部历史记忆。整体链路可以分为四个模块：

- 视觉/状态抽取模块：从 `obs` 中识别玩家中心 tile、宝箱、NPC、怪物、开关、按钮、墙体、桥方向和房间特征。
- 策略与规划模块：按任务维护阶段记忆，将任务拆成“拿钥匙”“按按钮”“切桥”“开宝箱”“击杀怪物”“进入出口”等子目标。

- **反馈记忆模块：**使用允许的 reward signals 与物品栏信息记录 key\_collected、door\_opened、monster\_killed、button\_pressed 等关键事件，避免依赖隐藏事件列表。
- **形式化验证模块：**在 Lean 中抽象环境的符号层语义，证明关键前置条件、不变式、完成后置条件、witness 计划可完成性，并对 task\_5 的高层阶段机策略给出动作合法性与目标可达性证明。

### 3.2 为什么选择该路线

选择该路线主要基于三点考虑。

- **可解释性：**五个任务均有清晰的目标链，规则阶段机比黑盒模型更容易定位失败原因。
- **与 Lean 对齐：**Lean 证明适合覆盖钥匙门、按钮门、桥状态、宝箱开启、怪物击杀和完成谓词这类符号语义，和阶段机策略天然一致。
- **实现风险可控：**课程时间有限，先做可运行 baseline，再把高层环境语义形式化，从头训练复杂策略更稳健。

当前阶段已经选择“统一 Policy 骨架 + 分任务逐步扩展 + 可验证层优先”的路线。具体而言，项目先在自有代码目录中建立统一的 Agent 入口，优先跑通 task\_1 的 baseline，再逐步扩展到 task\_2 至 task\_5，并以阶段机、合法动作筛选器和动态对象状态跟踪作为后续形式化重点。目前自有 baseline 已覆盖 task\_1 至 task\_5。其中 task\_2 仍属于调试用途的脚本化版本，而 task\_3、task\_4 和 task\_5 已经采用像素房间识别、玩家中心格定位、物品栏和 reward 历史驱动的规则状态机。Lean 部分首先完成五个任务的环境形式化与 witness 可完成性证明；在此基础上，task\_5 还额外形式化了与 Python 阶段机对应的高层符号策略，证明策略生成的房间级动作序列合法，并能达到全宝箱开启目标。

## 4 视觉与状态抽取

### 4.1 输入与输出设计

视觉模块输入为 RGB 像素帧 obs。输出不是完整地图重建，而是服务于规则策略的少量符号状态：

- **玩家中心 tile：**根据玩家绿色精灵像素反推精灵左上角，再取中心点所在 tile。
- **关键对象：**通过颜色计数识别宝箱、NPC、怪物、墙体、开关、按钮、深渊和桥。

- 房间类别：针对 task\_3、task\_4、task\_5 分别用房间内稳定对象和地形特征识别当前房间。
- 动态状态：task\_4 通过桥像素分布识别 west\_to\_north、west\_to\_east、west\_to\_south 三种桥方向；task\_5 通过按钮、宝箱和房间特征识别当前子任务位置。

## 4.2 实现方法

实现上采用基于颜色模板和 tile 统计的轻量规则方法。baseline\_policy.py 中定义了玩家、墙、宝箱、NPC、怪物、桥、开关和按钮的颜色常量，并提供 count\_color、count\_any\_color、tile\_has\_chest、tile\_has\_switch、tile\_has\_button 等辅助函数。

- 玩家定位不直接读取 info["agent"], 而是从像素中恢复中心 tile。
- 房间识别不直接读取 info["env"], 而是通过房间内宝箱、NPC、桥、开关、墙体开口等稳定视觉特征判断。
- 怪物、按钮、开关和宝箱状态通过像素与 reward 历史共同维护；例如 task\_5 用 button\_pressed、door\_opened、monster\_killed、chest\_opened 等 reward signals 更新内部记忆。
- 物品栏只用于读取课程允许的钥匙、装备等显式状态。

## 4.3 误差与局限

该视觉方案依赖当前地图素材的固定颜色和固定 tile 尺寸，因此更适合作为课程环境内的可解释 baseline，而不是通用视觉模型。主要风险包括：角色精灵中心与碰撞箱位置存在偏差、tile 已到位但碰撞箱仍未越过出口边界、以及对象被开启或按下后像素外观变化导致房间识别失败。针对这些问题，策略中加入了若干行/列对齐阶段，例如 task\_5 进入东门、从东房返回、进入西房前都保留额外对齐动作，以减少碰撞边界误差。

# 5 策略与规划

## 5.1 基线策略

基线策略采用规则式阶段机，而不是学习模型。

- task\_1：使用固定像素动作序列完成“取钥匙→北门出口”。
- task\_2：使用从参考成功轨迹抽取的脚本化动作序列，完成战斗、开箱和出口目标。

- **task\_3**: 使用像素识别的房间类别、玩家 tile 和钥匙状态，完成“西行到钥匙房→开箱→返回起点→开锁门”。
- **task\_4**: 维护桥方向和阶段记忆，按“北房钥匙→切桥到东房拿剑→切桥到南房杀守卫→回中心开最终宝箱”推进。
- **task\_5**: 维护按钮、钥匙、东门、怪物、四个宝箱等事件记忆，按“起始宝箱→按钮→南房钥匙→起始房杀怪→东房回血→西房金币”推进。

## 5.2 多任务扩展

多任务扩展的核心是把每个任务拆成少量稳定子目标，并在 `AgentHistory.notes` 中保存跨步记忆。对单房间任务，固定计划即可完成；对多房间任务，策略需要记录当前阶段、是否已开箱、是否已取得钥匙、桥是否已旋转目标方向、怪物是否已经击杀等信息。路径选择主要使用 tile 级贪心移动和固定 waypoint，在出口和交互点附近加入额外对齐动作，保证像素控制下的碰撞箱能够真正进入交互范围。

## 5.3 最终提交版本

当前提交版本与调试过程的区别如下：

- 调试阶段曾使用 `info` 校准房间、坐标和事件语义，便于定位失败原因。
- 当前 baseline 的 `task_3`、`task_4` 和 `task_5` 主体决策已转向像素识别、reward 历史和物品栏，不直接读取隐藏的 `info["env"]`、`info["agent"]`、`info["dynamic"]` 或 `info["events"]`。
- `task_1` 与 `task_2` 仍是固定动作序列，适合作为稳定 baseline，但泛化能力弱于后续的像素阶段机。
- Lean 当前验证的是符号层环境、显式 witness 计划，以及 `task_5` 高层阶段机策略的合法性与完成性；尚未证明像素感知层一定能在所有渲染扰动下恢复这些符号状态。

# 6 Lean 形式化与证明

## 6.1 形式化对象总览

本项目对 `task_1` 至 `task_5` 全部完成了 Lean 环境形式化与可完成性证明，对应文件位于 `student_agent/lean/` 目录下：

- `Task1EnvironmentFormalization.lean`: tile 级单房间钥匙门；

- `Task2EnvironmentFormalization.lean`: zone 级战斗与条件门;
- `Task3EnvironmentFormalization.lean`: room 级三房间往返取钥匙;
- `Task4EnvironmentFormalization.lean`: room 级旋转桥与隐藏宝箱;
- `Task5EnvironmentFormalization.lean`: room 级四房间按钮门与全宝箱收集。

五个文件均通过 Lean LSP 诊断 (零错误), 并按照统一的设计原则建模:

1. **抽象层级与 Python 行为对齐**: 根据每个任务的关键机制选择 tile / zone / room 抽象层级, 并与 `movement.py`、`interactions.py`、`combat.py` 及 `room_*.json` 中的语义保持一致;
2. **保留约束而不照搬实现**: 刻意忽略像素、碰撞盒、怪物 AI、击退等低层细节, 只保留影响规划正确性的状态字段、前置条件与事件触发;
3. **统一证明骨架**: 每个任务都给出“局部前置/后置引理  $\rightarrow$  必要性引理  $\rightarrow$  Reachable 归纳不变式  $\rightarrow$  `completion_postcondition`  $\rightarrow$  witness 计划  $\rightarrow$  可完成性”的完整证明链。

下面分任务展开。其中 `task_4` 因机制最复杂 (动态对象状态、条件门、关键道具获取和事件驱动显现), 作为重点展开对象。

## 6.2 Task 1: 单房间钥匙门形式化

`task_1` 是最简单的单房间取钥匙走北门任务。Lean 文件以 tile 级建模, 对应 `room_001.json` 的  $8 \times 10$  列网格。

状态与动作:

- `EnvState`: `x, y : Nat` (tile 坐标)、`keys`、`chestOpened`、`completed`;
- `Action`: `up / down / left / right / openChest / goNorth`;
- `isWalkable` 严格复刻 `room_001.json` 中的墙体布局 (row 2 的 `##..#####`、row 5 的 `#####...`);
- `isAdjacentToChest` 使用 Manhattan 距离  $\leq 1$ , 对齐 `state.py:is_adjacent` 的语义 (包含距离 0)。

关键约束:

- `openChest` 触发条件为邻接 chest tile 且未开过, 开箱后 `keys + 1`;
- `goNorth` 仅在 (4, 0) 且 `keys > 0` 时触发, 成功后消耗一把钥匙并将 `completed` 置真 (对应 `consume_key: true`)。



| 定理名称                                            | 形式化对象  | 结论说明                                                        |
|-------------------------------------------------|--------|-------------------------------------------------------------|
| <code>step_preserves_walkable</code>            | 安全性    | 任意动作后玩家仍位于可走 tile 上。                                        |
| <code>openChest_only_at_chest</code>            | 开箱前置条件 | 不邻接 chest 时 <code>openChest</code> 不改变状态。                   |
| <code>openChest_at_chest_increases_keys</code>  | 开箱后置条件 | 邻接且未开过时, <code>keys</code> 增加 1。                            |
| <code>goNorth_only_at_exit</code>               | 北门前置条件 | 不在 (4, 0) 时 <code>goNorth</code> 无效。                        |
| <code>goNorth_without_key</code>                | 北门前置条件 | 在出口但无钥匙时 <code>goNorth</code> 无效。                           |
| <code>goNorth_at_exit_with_key_completes</code> | 北门后置条件 | 在出口且持钥匙时 <code>completed</code> 置真。                         |
| <code>only_goNorth_sets_completed</code>        | 必要性引理  | <code>goNorth</code> 是唯一能把 <code>completed</code> 从假翻为真的动作。 |
| <code>witnessPlan_reaches_goal</code>           | 执行轨迹   | 22 步 witness 计划可达目标。                                        |
| <code>task1_completable</code>                  | 可完成性   | <code>task_1</code> 在该模型下可完成。                               |

**witness 计划:** tile 级路径

$$(4, 6) \xrightarrow{\text{right} \times 3} (7, 6) \xrightarrow{\text{up} \times 3} (7, 3) \xrightarrow{\text{left} \times 7} (0, 3) \xrightarrow{\text{openChest}} . \xrightarrow{\text{right} \times 3, \text{up} \times 3, \text{right}} (4, 0) \xrightarrow{\text{goNorth}} \text{goal}.$$

由于 22 步计划超出 `decide` 递归深度,按项目约定使用 `native_decide` 验证 `witnessPlan_executes_t`

### 6.3 Task 2: 战斗与条件门形式化

`task_2` 在单房间内引入 `chaser` 怪物、HP、陷阱与西向条件门。为突出战斗与条件门语义,采用 zone 级抽象。

**状态与动作:**

- `Zone`: `safeCenter` / `nearMonster` / `nearChest` / `westExit` / `topTrap` / `bottomTrap`;
- `EnvState`: `zone`、`hp`、`monsterAlive`、`monsterHp`、`chestOpened`、`keys`、`completed`;
- `Action`: `moveTo z` / `attack` / `openChest` / `useExit` / `wait`。

**关键约束:**

- `attack` 仅在 `nearMonster`、怪物存活且 `hp > 1` 时生效；`monsterHp` 为 1 时击杀（置 `monsterAlive := false`），否则仅 `monsterHp - 1`；
- `useExit` 在 `westExit` 且怪物已死且 `keys > 0` 时触发，成功后不消耗钥匙（对应 `requires` 中未设 `consume_key`）；
- 走入 `topTrap` / `bottomTrap` 时 HP 减 1 并被重置回 `safeCenter`。

| 定理名称                                                 | 形式化对象   | 结论说明                                           |
|------------------------------------------------------|---------|------------------------------------------------|
| <code>trap_move_reduces_hp</code>                    | 陷阱语义    | 踩陷阱后 <code>hp</code> 减少 1。                     |
| <code>safe_move_preserves_*</code>                   | 安全移动不变式 | 非陷阱 <code>moveTo</code> 保持 HP/怪物/宝箱/钥匙/完成状态不变。 |
| <code>attack_reduces_monsterHp_when_adjacent</code>  | 战斗语义    | 邻接且存活且 HP 充足时， <code>monsterHp</code> 减 1。     |
| <code>attack_kills_monster_when_monsterHp_one</code> | 战斗语义    | <code>monsterHp = 1</code> 时一击击杀。              |
| <code>attack_no_effect_*</code>                      | 战斗前置条件  | 不在 <code>nearMonster</code> 、怪物已死或 HP 过低时攻击无效。 |
| <code>exit_blocked_if_monster_alive</code>           | 条件门     | 怪物存活时 <code>useExit</code> 无效。                 |
| <code>exit_blocked_without_key</code>                | 条件门     | 无钥匙时 <code>useExit</code> 无效。                  |
| <code>exit_succeeds_after_requirements</code>        | 条件门后置   | 怪物已死且持钥匙时 <code>completed</code> 置真。           |
| <code>only_useExit_sets_completed</code>             | 必要性引理   | <code>useExit</code> 是唯一完成动作。                  |
| <code>completion_postcondition</code>                | 完成后置    | 任意可达完成态满足怪物已死 $\wedge$ 宝箱已开。                   |
| <code>hp_non_increasing</code>                       | 安全性     | 任意动作后 HP 不增。                                   |
| <code>witnessPlan_reaches_goal</code>                | 执行轨迹    | 7 步 <code>witness</code> 计划可达目标。               |
| <code>task2_completable</code>                       | 可完成性    | <code>task_2</code> 在该模型下可完成。                  |

**witness 计划：** zone 级路径

`safeCenter`  $\rightarrow$  `nearMonster`  $\xrightarrow{\text{attack} \times 2} \cdot \rightarrow$  `nearChest`  $\xrightarrow{\text{openChest}} \cdot \rightarrow$  `westExit`  $\xrightarrow{\text{useExit}} \text{goal}$ .

## 6.4 Task 3: 多房间往返形式化

task\_3 是首个多房间任务：从 start\_room 出发向西穿过 monster\_hall 到达 key\_room，取钥匙后返回 start\_room 走东向锁定出口。采用 room 级抽象。

状态与动作：

- Room: startRoom / monsterHall / keyRoom;
- EnvState: room、keys、keyChestOpened、hallMonsterAlive、completed;
- Action: goWest / goEast / openChest / openLockedExit / wait。

关键约束：

- goWest: startRoom  $\rightarrow$  monsterHall  $\rightarrow$  keyRoom; goEast: 反向; keyRoom 的 goWest 与 startRoom 的 goEast 无效;
- openChest 仅在 keyRoom 且未开过时生效，开箱后 keys + 1;
- openLockedExit 仅在 startRoom 且 keys > 0 时触发，成功后消耗钥匙并完成（对应 consume\_key: true）。

| 定理名称                                      | 形式化对象 | 结论说明                                 |
|-------------------------------------------|-------|--------------------------------------|
| west_then_west_reaches_keyRoom            | 导航语义  | 从 startRoom 连续两次 goWest 可达 keyRoom。  |
| openChest_only_in_keyRoom                 | 开箱前置  | 不在 keyRoom 时 openChest 无效。           |
| openChest_in_keyRoom_increases_keys       | 开箱后置  | 在 keyRoom 且未开过时 keys 增 1。            |
| openLockedExit_blocked_without_key        | 锁门前置  | 在 startRoom 但无钥匙时 openLockedExit 无效。 |
| openLockedExit_consumes_key_and_completes | 锁门后置  | 持钥匙时消耗一把钥匙并完成。                       |
| hallMonsterAlive_never_changes            | 不变式   | Task 3 无攻击动作，hallMonsterAlive 恒不变。   |
| only_openLockedExit_sets_completed        | 必要性引理 | openLockedExit 是唯一完成动作。              |
| keyChestOpened_or_keys_zero               | 归纳不变式 | 任意可达状态：宝箱已开 $\vee$ 钥匙数为 0。           |

| 定理名称                     | 形式化对象 | 结论说明                                           |
|--------------------------|-------|------------------------------------------------|
| completion_postcondition | 完成后置  | 任意可达完成态满足 <code>keyChestOpened = true</code> 。 |
| witnessPlan_reaches_goal | 执行轨迹  | 6 步 witness 计划可达目标。                            |
| task3_completable        | 可完成性  | <code>task_3</code> 在该模型下可完成。                  |

witness 计划:

$$\text{startRoom} \xrightarrow{\text{goWest} \times 2} \text{keyRoom} \xrightarrow{\text{openChest}} \cdot \xrightarrow{\text{goEast} \times 2} \text{startRoom} \xrightarrow{\text{openLockedExit}} \text{goal}.$$

由于计划简单, `witnessPlan_executes_to_finalState` 用 `rfl` 直接验证。

## 6.5 Task 4: 旋转桥与隐藏宝箱形式化

`task_4` 引入旋转桥、条件门、装备获取与事件驱动显现机制, 是机制最复杂的一关, 也是本报告的重点展开对象。对应 Lean 文件为 `Task4EnvironmentFormalization.lean`, 与 Python 环境中以下模块对齐: `movement.py` (门与房间切换)、`interactions.py` (开关与宝箱)、`combat.py` (击杀怪物后揭示宝箱)。

具体抽象:

- 房间状态: 使用 `Room` 枚举描述五个房间 `west / center / north / east / south`。
- 动态对象状态: 使用 `BridgeState` 描述中央桥的三种连通状态 `westToNorth / westToEast / westToSouth`。
- 玩家资源与关键事件: 记录 `keys`、`hasSword`、`hasShield`, 以及北房宝箱、东房宝箱、南房守卫和中央终点宝箱的状态。
- 动作: 抽象为房间级动作, 包括转桥、进出中心房、开宝箱、攻击和开启最终宝箱。
- 转移函数: 用确定性函数 `step` 描述状态更新。它保留了三类关键约束: 东门需要钥匙但不消耗钥匙 (对应 `consume_key: false`); 桥状态决定中心房可达方向; 南房守卫被击杀后才会揭示中央最终宝箱。
- 目标谓词: `GoalReached` 定义为最终宝箱已开启。

| 定理名称                     | 形式化对象 | 结论说明                      |
|--------------------------|-------|---------------------------|
| rotateBridge_three_cycle | 桥状态机  | 证明西房开关连续触发三次后, 桥状态回到初始方向。 |

| 定理名称                                                   | 形式化对象   | 结论说明                                                                                      |
|--------------------------------------------------------|---------|-------------------------------------------------------------------------------------------|
| <code>goEast_blocked_without_key</code>                | 东门条件    | 证明当玩家位于中心房、桥已连到东侧但没有钥匙时, <code>goEast</code> 不产生状态变化。                                     |
| <code>goEast_succeeds_with_key</code>                  | 东门条件    | 证明在中心房、桥连东侧且已持钥匙时, <code>goEast</code> 能把玩家带入东房。                                          |
| <code>goEast_preserves_keys</code>                     | 东门不变式   | 证明 <code>goEast</code> 不消耗钥匙 (Python <code>consume_key: false</code> 对齐)。                 |
| <code>attack_without_sword_has_no_effect</code>        | 战斗语义    | 证明没有剑时, 在南房执行攻击不会击杀守卫, 也不会揭示最终宝箱。                                                         |
| <code>attack_with_sword_reveals_final_chest</code>     | 战斗与事件触发 | 证明持剑击败南房守卫后, <code>guardianAlive</code> 变为假, 且 <code>finalChestVisible</code> 变为真。        |
| <code>openFinalChest_requires_visibility</code>        | 终点交互语义  | 证明若最终宝箱尚未显现, 则在中心房尝试开启最终宝箱不会成功。                                                           |
| <code>only_openFinalChest_sets_finalChestOpened</code> | 必要性引理   | <code>openFinalChest</code> 是唯一完成动作。                                                      |
| <code>northChestOpened_or_keys_zero</code>             | 归纳不变式   | 任意可达状态: 北房宝箱已开 $\vee$ 钥匙数为 0。                                                             |
| <code>finalChestVisible_implies_guardianDead</code>    | 归纳不变式   | 任意可达状态: 终点宝箱显现 $\Rightarrow$ 守卫已死。                                                        |
| <code>completion_postcondition</code>                  | 完成后置    | 任意可达完成态满足 <code>guardianAlive = false</code> 。                                            |
| <code>witnessPlan_reaches_goal</code>                  | 执行轨迹    | 给出一条具体计划, 证明从初始状态出发可以完成“取钥匙 $\rightarrow$ 拿剑 $\rightarrow$ 击杀守卫 $\rightarrow$ 开最终宝箱”的任务链。 |
| <code>task4_completable</code>                         | 可达性     | 由上述执行轨迹推出: 在该形式化模型下, <code>task_4</code> 是可完成的。                                           |

**witness 计划：17 步房间级路径**

$$\begin{aligned}
 &\text{west} \xrightarrow{\text{goCenter, goNorth, openChest}} \text{north} \xrightarrow{\text{goCenter, goWest, toggleBridge}} \text{west} \\
 &\quad \xrightarrow{\text{goCenter, goEast, openChest}} \text{east} \xrightarrow{\text{goCenter, goWest, toggleBridge}} \text{west} \\
 &\quad \xrightarrow{\text{goCenter, goSouth, attack}} \text{south} \xrightarrow{\text{goCenter, openFinalChest}} \text{goal}.
 \end{aligned}$$

witnessPlan\_executes\_to\_finalState 用 rfl 直接验证。

## 6.6 Task 5：四房间按钮门与全宝箱收集形式化

### 6.6.1 环境建模

Task 5 包含 4 个房间 (startRoom / southRoom / eastRoom / westRoom)，关键机制为：

- 按钮门：startRoom 的南出口为条件门 (button\_pressed)，初始 buttonPressed = false;
- 钥匙门：startRoom 的东出口为锁门 (consume\_key = true)，初始 keys = 0;
- 差异化宝箱：4 个房间各有 1 个宝箱，效果不同——startRoom (金币，无副作用)、southRoom (钥匙, keys + 1)、eastRoom (回血, hp + 1)、westRoom (金币，无副作用)；
- 世界完成：与 task\_1--4 不同，task\_5 无 complete\_task 出口，世界完成条件为 all\_chests\_opened (对齐 progress.py:36)。

GoalReached 定义为 allChestsOpened，即四个宝箱全部打开。初始状态：room = startRoom, buttonPressed = false, keys = 0, hp = 5，四个 ChestOpened 均为 false。

### 6.6.2 状态空间与转移函数

- pressButton：仅在 startRoom 生效，buttonPressed := true;
- goSouth：需 room = startRoom ∧ buttonPressed = true，进入 southRoom;
- goNorth：需 room = southRoom，返回 startRoom;
- goEast：从 startRoom (keys > 0) 进入 eastRoom 并消耗钥匙；从 westRoom 返回 startRoom (不消耗钥匙)；
- goWest：从 startRoom 进入 westRoom；从 eastRoom 返回 startRoom;
- openChest：按房间分支，每房间最多开一次，效果见上。

### 6.6.3 核心定理

#### 6.6.4 归纳不变式

`buttonPressed_false_implies_not_in_south_and_chest_closed` 是核心不变式：若 `buttonPressed = false`, 则玩家不在 `southRoom` 且南宝箱未开。其逆否命题 `southChestOpened_implies_buttonPressed` 说明南宝箱开启蕴含按钮已按。

`southChestClosed_implies_keys_zero` 利用归纳法证明：南宝箱是唯一的钥匙来源，而东出口（唯一消耗钥匙的动作）需要 `keys > 0`，因此在南宝箱未开之前不可能存在任何钥匙。

#### 6.6.5 Witness 计划

10 步计划：

$$\begin{aligned}
 \text{start} &: \xrightarrow{\text{openChest}} \text{start} \xrightarrow{\text{pressButton}} \text{start} \xrightarrow{\text{goSouth}} \text{south} \\
 &\xrightarrow{\text{openChest (keys+1)}} \text{south} \xrightarrow{\text{goNorth}} \text{start} \xrightarrow{\text{goEast (keys-1)}} \text{east} \\
 &\xrightarrow{\text{openChest (hp+1)}} \text{east} \xrightarrow{\text{goWest}} \text{start} \xrightarrow{\text{goWest}} \text{west} \xrightarrow{\text{openChest}} \text{goal}.
 \end{aligned}$$

终态: `room = westRoom, buttonPressed = true, keys = 0, hp = 6`, 四个 `ChestOpened` 均为 `true`。 `witnessPlan_executes_to_finalState` 用 `rfl` 直接验证。

#### 6.6.6 策略形式化与正确性证明

为覆盖评分项中的“策略形式化与证明”，`Task5EnvironmentFormalization.lean` 在环境模型之后进一步定义了 `PolicyStage` 与 `symbolicPolicy`。 `PolicyStage` 把任务拆成 `openStartChest`、`pressStartButton`、`openSouthChest`、`openEastChest`、`openWestChest` 和 `done` 六个阶段； `symbolicPolicy` 根据当前房间、按钮状态、钥匙数和四个宝箱状态输出下一步房间级动作。

为了证明策略不是单纯列出路径，Lean 文件还定义了：

- `policyTrace`: 从当前状态反复调用 `symbolicPolicy` 生成动作轨迹；
- `runPolicy`: 直接执行策略得到后继状态；
- `ActionEnabled`: 刻画每个动作在当前状态下是否合法，例如 `goSouth` 需要按钮已按， `goEast` 需要钥匙或处于西房返回分支；
- `actionsEnabledAlong`: 递归检查整条策略轨迹上每一步动作都满足对应前置条件。

核心策略定理如下：

```

policyTrace_10_matches_witnessPlan :
  policyTrace 10 initialState = witnessPlan;
policyTrace_10_actions_enabled :
  actionsEnabledAlong initialState (policyTrace 10 initialState);
symbolicPolicy_run_10_finalState :
  runPolicy 10 initialState = finalState;
task5_symbolicPolicy_completes :
   $\exists n$ , GoalReached (runPolicy n initialState).

```

这说明 task\_5 的 Lean 部分不再只证明“存在一条完成路径”，而是证明一个显式阶段机策略从初态出发会选择合法动作，并在有限步内完成目标。

## 6.7 证明统一思路

五个任务遵循同一套证明骨架，便于复用与审阅：

1. **局部前置/后置引理**：对每个动作给出“何时触发”和“触发后字段如何变化”。这一步对应评分细则中的“状态 / 动作 / 转移是否定义完整、是否能证明基本合法性与安全性”。例如 Task 4 的 `goEast_blocked_without_key` 直接对应 Python `movement.py` 对 `locked_key` 出口的检查；`attack_with_sword_reveals_final_chest` 对应 `combat.py` 中“清怪后揭示宝箱”的事件语义；`rotateBridge_three_cycle` 对应 `interactions.py` 中 `switch` 对 `center_bridge` 的循环切换。
2. **必要性引理** (`only_X_sets_completed`)：证明只有某个特定动作能把完成标志从假翻为真。通过 `non_X_preserves_completed` 这一“其它动作保持完成字段不变”的引理来支撑。
3. **Reachable 归纳不变式**：定义归纳谓词 `Reachable`，对初始状态和任意 `step` 闭合；证明诸如“未开箱则钥匙数为 0”“终点宝箱显现则守卫已死”等性质在所有可达状态上成立。
4. **completion\_postcondition**：在 `Reachable` 归纳之上，证明“任意可达完成态必然满足某组前置条件”（如 Task 2 中“怪物已死  $\wedge$  宝箱已开”、Task 4 中“守卫已死”）。证明中显式调用必要性引理与归纳不变式。
5. **witness 计划**：在局部规则已经明确的基础上，构造一条显式的见证计划 `witnessPlan`。由于 `step` 被定义为确定性函数，因此整条轨迹的正确性可以通过归约直接验证（简单计划用 `rfl`，长计划用 `native_decide`）。



6. 可完成性：由 `witnessPlan_reaches_goal` 直接推出 `taskN_completable`，即“存在计划使目标谓词成立”。

这使得“任务可完成”从口头分析转化为可检查的形式化结论，并且完成态的必要前置条件也被显式记录下来。

## 6.8 证明局限与边界

当前环境形式化刻画的是 `task_1` 至 `task_5` 的 `tile / zone / room` 级符号语义，而不是 Python 引擎的逐像素精确副本，因此存在以下边界：

- 未形式化像素级移动、碰撞盒、路径细节与房间内精确坐标（`task_1` 保留了 `tile` 级网格，但 `task_2` 至 `task_5` 均采用更高的 `zone / room` 抽象）。
- 未形式化怪物 AI、击退、受伤时序和 `shield` 格挡的连续动力学。
- 未证明视觉模块一定能从 `obs` 中稳定恢复这些符号状态；该部分属于感知层，而不是当前 Lean 环境模型的覆盖范围。
- 当前证明的是一条显式计划的可执行性和若干局部环境性质，还没有证明“任意合法 planner 都能找到该计划”这类更强的完备性命题。

这一边界是有意为之：课程要求强调 Lean 证明应覆盖“可验证层”。对于本项目而言，最具价值且最容易和实际代码保持一致的对象，正是钥匙/门条件、战斗与事件触发、桥状态机与任务完成谓词这类符号层环境语义。

# 7 实验设计与结果分析

## 7.1 实验设置

实验使用仓库自带评测脚本：

```
python utils/evaluate_policy.py --policy
student_agent/baseline_policy.py --num-envs 1
```

运行环境为 Python 3.12 虚拟环境 `.venv312`。本次报告记录的是 2026 年 7 月 6 日在本地工作区重新运行的单 seed 回归结果。

- 测试任务：`mathematical_logic/task_1` 至 `mathematical_logic/task_5`。
- 运行次数：每个任务 1 个 episode，`seed = 0`。
- 成功率统计方式：评测脚本输出的 `success_rate`。

- 策略版本: `student_agent/baseline_policy.py`。
- Lean 检查: 逐文件运行 `lean student_agent/lean/*.lean` 对 6 个 Lean 文件进行编译检查。

## 7.2 实验结果

| 任务     | 方法版本                   | 结果指标                                            | 备注          |
|--------|------------------------|-------------------------------------------------|-------------|
| Task 1 | 固定动作 baseline          | success = 1.000; steps = 290; reward = 127.050  | 单房间钥匙门      |
| Task 2 | 参考轨迹脚本 baseline        | success = 1.000; steps = 182; reward = 126.180  | 战斗与开箱       |
| Task 3 | 像素房间识别 + 规则状态机         | success = 1.000; steps = 525; reward = 159.750  | 跨房间取钥匙      |
| Task 4 | 像素桥识别 + 显式阶段机          | success = 1.000; steps = 1089; reward = 249.110 | 桥、剑、守卫、最终宝箱 |
| Task 5 | 像素识别 + reward 记忆 + 阶段机 | success = 1.000; steps = 1112; reward = 155.730 | 四房间全宝箱      |

表 6: 各任务实验结果汇总

| 检查对象                               | 结果 | 说明                  |
|------------------------------------|----|---------------------|
| KillMonsterFormalization.lean      | 通过 | 怪物击杀示例证明            |
| Task1EnvironmentFormalization.lean | 通过 | Task 1 环境形式化        |
| Task2EnvironmentFormalization.lean | 通过 | Task 2 环境形式化        |
| Task3EnvironmentFormalization.lean | 通过 | Task 3 环境形式化        |
| Task4EnvironmentFormalization.lean | 通过 | Task 4 环境形式化        |
| Task5EnvironmentFormalization.lean | 通过 | Task 5 环境形式化与高层策略证明 |

表 7: Lean 文件逐文件检查结果

### 7.3 结果分析

从本次单 seed 回归看，五个任务均能由当前 baseline 完成。task\_1 和 task\_2 的完成步数较少，主要因为采用了固定动作序列；task\_3 至 task\_5 的步数更高，是因为策略在出口和交互点附近加入了保守对齐动作，以换取更稳定的像素控制。

- 当前成功率最高的结论只限于本地单 seed 回归，不能直接推出多 seed 或地图扰动下的泛化性能。
- 主要风险来自视觉定位与碰撞箱偏差，而不是高层目标链错误。
- reward signals 对跨房间记忆非常有帮助，尤其是按钮、开门、击杀和开箱事件。
- Lean 证明与实验互补：实验说明 Python baseline 可运行，Lean 证明说明抽象后的环境语义、witness 计划以及 task\_5 高层阶段机策略在逻辑上自洽。

## 8 项目分工与推进过程

### 8.1 成员分工

由于报告模板尚未记录真实姓名，表中先按角色记录可替换分工。最终提交前需要由小组将“成员待替换”替换为真实姓名。

| 成员        | 负责内容                | 主要产出                                     |
|-----------|---------------------|------------------------------------------|
| 组长（待替换）   | 项目组织、报告统筹、最终集成      | 报告框架、实验结果整理、提交检查                         |
| 成员 1（待替换） | Python baseline 与评测 | <code>baseline_policy.py</code> 、五任务运行结果 |
| 成员 2（待替换） | Lean Task 1-3 形式化   | 钥匙门、战斗、跨房间可达性证明                          |
| 成员 3（待替换） | Lean Task 4-5 形式化   | 桥状态、按钮门、全宝箱完成证明                          |
| 成员 4（待替换） | 文档、截图与复核            | 项目管理文档、报告校对、实验记录                         |

表 8: 小组分工

## 8.2 时间安排

- **2026-07-03:** 阅读课程说明与仓库结构，建立项目管理文档和报告骨架；安装依赖，跑通示例策略；创建 `student_agent/` 和统一 `baseline` 入口；初步覆盖 `task_1` 至 `task_4`。
- **2026-07-05:** 将 `task_3`、`task_4` 改造为像素感知版本；修正玩家中心 `tile` 识别；接入 `task_5`；完成五任务单 `seed` 回归；逐文件检查 Lean 文件。
- **2026-07-06:** 同步团队更新，补全 `task_1` 至 `task_5` 的 Lean 环境形式化与可完成性证明；重新运行 Python evaluation 和全部 Lean 检查；进一步补充 `task_5` 高层策略形式化与合法性/完成性证明；把最终实验结果与证明边界写入报告。

## 9 总结与展望

本文完成了 NesyLink 数理逻辑课程项目的三条主线工作。第一，构建了统一的 Python baseline，并在 `task_1` 至 `task_5` 的本地单 `seed` 评测中全部成功。第二，使用 Lean 对五个任务的环境语义进行了符号层形式化，覆盖状态、动作、转移、关键前置条件、不变式、完成后置条件和 witness 可完成性证明；并对 `task_5` 的阶段机策略补充了动作合法性与目标完成性证明。第三，将实验结果、实现路线和形式化边界整理进报告，明确区分了调试信息、提交版策略输入和 Lean 证明对象。

当前工作的主要不足是：Lean 已证明 `task_5` 的高层符号策略，但尚未逐行证明 Python 像素 `waypoint` 执行代码等价于该符号策略，也没有证明视觉模块一定能在所有渲染扰动下稳定恢复符号状态；Python baseline 在 `task_1` 和 `task_2` 上仍偏脚本化，泛

化能力有限。如果时间更充足，可以进一步把 `task_3` 和 `task_4` 也补成同等强度的高层策略形式化；也可以增加多 seed 回归、失败截图和更系统的视觉鲁棒性测试。

表 5: Task 5核心定理一览

| 定理                                                            | 作用                                     |
|---------------------------------------------------------------|----------------------------------------|
| pressButton_only_in_startRoom                                 | 按钮仅起点房生效                               |
| goSouth_blocked_without_<br>button                            | 条件门前置                                  |
| goEast_blocked_without_key                                    | 锁门前置                                   |
| goEast_from_start_consumes_<br>key                            | 东出口消耗钥匙                                |
| openChest_south_grants_key                                    | 南房宝箱给钥匙                                |
| openChest_east_heals                                          | 东房宝箱回血                                 |
| only_pressButton_changes_<br>buttonPressed                    | 必要性: 按钮                                |
| non_goEast_non_openChest_<br>preserves_keys                   | 必要性: 钥匙                                |
| buttonPressed_false_implies_<br>not_in_south_and_chest_closed | 归纳不变式                                  |
| southChestOpened_implies_<br>buttonPressed                    | 南宝箱开 $\Rightarrow$ 按钮已按                |
| southChestClosed_implies_<br>keys_zero                        | 钥匙来源不变式                                |
| completion_postcondition                                      | 完成态 $\Rightarrow$ buttonPressed = true |
| task5_completable                                             | 可完成性                                   |
| policyTrace_10_matches_<br>witnessPlan                        | 策略轨迹与 witness 计划一致                     |
| policyTrace_10_actions_<br>enabled                            | 策略每一步动作均满足前置条件                         |
| symbolicPolicy_run_10_<br>finalState                          | 策略执行 10 步到达终态                          |
| task5_symbolicPolicy_<br>completes                            | 阶段机策略完成全宝箱目标                           |